

UNIVERSITÀ DEGLI STUDI MILANO-BICOCCA

APPUNTI DEL CORSO DI

Analisi e Progettazione del Software

Scritto da:

Nicholas Scorrano

Anno Accademico 2025/2026

Indice

1	Modello a cascata	5
1.1	Cosa è?	5
1.2	Fasi del modello a cascata	5
1.3	Vantaggi	5
1.4	Svantaggi	5
2	UP - Unified Process	7
2.1	Cosa è?	7
2.2	Caratteristiche fondamentali	7
2.3	Fasi del processo	7
2.4	Ciclo di vita	7
2.5	Vantaggi	8
2.6	Svantaggi	8
3	Agile	10
3.1	Cosa è?	10
3.2	Caratteristiche	10
3.3	Manifesto	10
3.4	Vantaggi	10
3.5	Svantaggi	10
4	SCRUM	12
4.1	Cosa è?	12
4.2	Caratteristiche	12
4.2.1	Concetti Chiave	12
4.3	Ruoli principali	12
4.4	Ciclo di lavoro	13
4.5	Artefatti	13
4.6	Vantaggi	13
4.7	Svantaggi	14
5	Pattern GRASP	15
5.1	Definizione	15
5.1.1	Approccio RDD	15
5.1.2	Principi fondamentali	15
5.1.3	Relazione tra gli elaborati	16
5.2	Rappresentazione di un Pattern GRASP	16
6	Pattern GRASP di base	17
6.1	Creator	17
6.1.1	Problema	17
6.1.2	Soluzione	17
6.1.3	Benefici	17
6.1.4	Controindicazioni	17
6.1.5	Flusso di lavoro logico	18
6.1.6	Modello di dominio	18
6.1.7	Modello di sequenza	18
6.1.8	Diagrammi delle classi di progetto	19

6.2	Information Expert	19
6.2.1	Problema	19
6.2.2	Soluzione	19
6.2.3	Benefici	19
6.2.4	Controindizioni	20
6.2.5	Flusso di lavoro	20
6.2.6	Diagramma di sequenza	20
6.2.7	Diagramma delle classi	20
6.3	Low Coupling	21
6.3.1	Problema	21
6.3.2	Soluzione	21
6.3.3	Benefici	21
6.3.4	Controindicazioni	21
6.3.5	Flusso di lavoro	21
6.4	High Cohesion	22
6.4.1	Problema	22
6.4.2	Soluzione	22
6.4.3	Benefici	22
6.4.4	Controindicazioni	22
6.4.5	Flusso di lavoro	23
6.5	Controller	23
6.5.1	Problema	23
6.5.2	Soluzione	23
6.5.3	Benefici	24
6.5.4	Controindicazioni	24
6.5.5	Flusso di lavoro	24
7	Pattern GRASP avanzati	25
7.1	Pure Fabrication	25
7.1.1	Problema	25
7.1.2	Soluzione	25
7.1.3	Benefici	25
7.1.4	Controindicazioni	25
7.1.5	Flusso di lavoro	25
7.2	Polymorphism	26
7.2.1	Problema	26
7.2.2	Soluzione	26
7.2.3	Benefici	26
7.2.4	Controindicazioni	26
7.2.5	Flusso di lavoro	26
7.3	Indirection	27
7.3.1	Problema	27
7.3.2	Soluzione	27
7.3.3	Benefici	27
7.3.4	Controindicazioni	27
7.3.5	Flusso di lavoro	27
7.4	Protected Variations	28
7.4.1	Problema	28
7.4.2	Soluzione	28
7.4.3	Benefici	28
7.4.4	Controindicazioni	28
7.4.5	Flusso di lavoro	28

8	Design Pattern GoF	29
8.1	Cosa sono?	29
8.2	Confronto con i pattern GRASP	29
8.2.1	Principi contro Strutture	29
8.3	Adapter	30
8.3.1	Problema	30
8.3.2	Soluzione	30
8.3.3	Benefici	30
8.3.4	Controindicazioni	30
8.3.5	Implementazione	30
8.3.6	Dettagli	30
8.4	Factory	31
8.4.1	Problema	31
8.4.2	Soluzione	31
8.4.3	Benefici	31
8.4.4	Implementazione	31
8.4.5	Dettagli	31
8.5	Singleton	32
8.5.1	Problema	32
8.5.2	Soluzione	32
8.5.3	Benefici	32
8.5.4	Controindicazioni	32
8.5.5	Implementazione	32
8.5.6	Dettagli	32
8.6	Strategy	33
8.6.1	Problema	33
8.6.2	Soluzione	33
8.6.3	Benefici	33
8.6.4	Implementazione	33
8.6.5	Dettagli	33
8.7	Composite	34
8.7.1	Problema	34
8.7.2	Soluzione	34
8.7.3	Benefici	34
8.7.4	Implementazione	34
8.7.5	Dettagli	34
8.8	Facade	35
8.8.1	Problema	35
8.8.2	Soluzione	35
8.8.3	Benefici	35
8.8.4	Implementazione	35
8.8.5	Dettagli	35
8.9	Observer (Publish-Subscribe)	36
8.9.1	Problema	36
8.9.2	Soluzione	36
8.9.3	Benefici	36
8.9.4	Implementazione	36
8.9.5	Dettagli	36
9	Visibilità	37
9.1	Definizione	37
9.2	Tipi di visibilità	37
9.2.1	Visibilità per attributo	37

9.2.2	Visibilità per parametro	37
9.2.3	Visibilità locale	38
9.2.4	Visibilità globale	38
9.3	Trasformazione della visibilità	38
9.3.1	Da visibilità locale a visibilità per parametro	38
9.3.2	Da visibilità per parametro a visibilità per attributo	38
10	TDD - Test Driven Development	39
10.1	Definizione	39
10.2	Cosa è un test unitario?	39
10.2.1	Schema di test	39
10.2.2	Tipi di test	39
10.3	Vantaggi	39
10.4	Svantaggi	40
10.5	Progettazione degli input	40
11	Refactoring	41
11.1	Definizione	41
11.2	Obiettivi e Vantaggi	41
11.3	Quando usarlo?	41
11.4	Procedura	41
11.5	Cosa è un Code Smell?	42
11.6	Tabella Code Smell	42

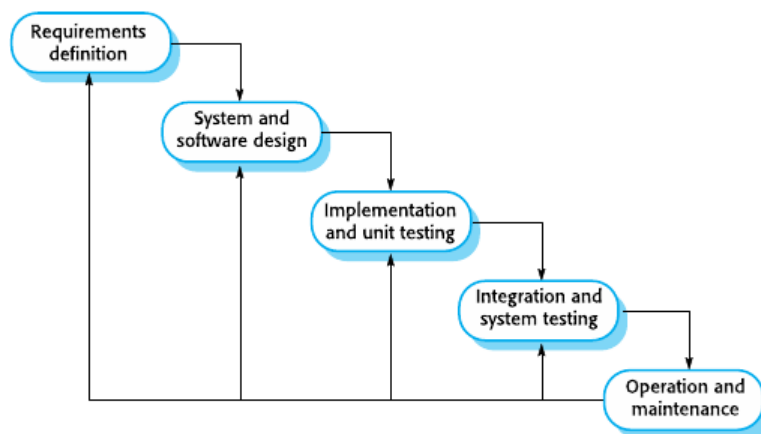
1. Modello a cascata

1.1 Cosa è?

Il modello a cascata è un classico modello di processo per lo sviluppo del software che si basa su un approccio rigorosamente guidato dal piano

La sua caratteristica fondamentale è la suddivisione del progetto in fasi separate e distinte.

La regola di base di questo modello impone che una fase debba essere completamente terminata prima di poter passare a quella successiva



1.2 Fasi del modello a cascata

1. Analisi e definizione dei requisiti
2. Progettazione del sistema e del software
3. Implementazione e test delle unità
4. Integrazione e test di sistema
5. Funzionamento e manutenzio

1.3 Vantaggi

- **Coordinamento di grandi progetti**

La natura rigidamente pianificata di questo modello aiuta a coordinare il lavoro in modo efficiente nei progetti di ingegneria dei sistemi di grandissime dimensioni

- **Sviluppo distribuito**

Si rivela particolarmente utile quando il sistema deve essere sviluppato in diversi siti o sedi geografiche

1.4 Svantaggi

- **Difficoltà nell'accogliere i cambiamenti**

Il principale svantaggio è l'incapacità di gestire modifiche ai requisiti una volta che il processo di sviluppo è stato avviato

- **Eccessiva inflessibilità**

Il modello impone, in linea di principio, che una fase debba essere completamente terminata prima di passare a quella successiva.

Questa rigida suddivisione rende estremamente difficile rispondere alle mutevoli esigenze dei clienti in corso d'opera

- **Soggetto a frequenti fallimenti**

Affinchè il modello a cascata abbia successo, i requisiti devono essere perfettamente compresi fin dall'inizio e subire modifiche molto limitate.

Poiché pochi sistemi aziendali godono di requisiti realmente stabili, l'applicazione di questo processo porta frequentemente al fallimento del progetto.

2. UP - Unified Process

2.1 Cosa è?

L'UP è un processo iterativo molto diffuso, concepito specificamente per lo sviluppo di software orientato agli oggetti.

È un framework estremamente flessibile e aperto, che incoraggia l'integrazione di pratiche provenienti da altri metodi agili

2.2 Caratteristiche fondamentali

- **Pilotato dai casi d'uso (e requisiti)**

Lo sviluppo è guidato dalle reali necessità degli utenti, coinvolgendoli continuamente per ottenere valutazioni e feedback

- **Guidato dai fattori di rischio**

Affronta le problematiche a maggior rischio e a più alto valore aziendale fin dalle primissime iterazioni

- **Incentrato sull'architettura**

Le fasi iniziali mirano a costruire, testare e stabilizzare un nucleo architettonico coeso prima di sviluppare il resto del sistema

2.3 Fasi del processo

Un progetto UP organizza il proprio ciclo di vita suddividendo il tempo in quattro macro-fasi sequenziali:

1. **Ideazione (Inception)**

È l'avvio del progetto.

In questa fase si definisce una visione approssimativa, si valuta la portata e si abbozzano le prime stime di costi e tempi

2. **Elaborazione (Elaboration)**

Si realizza il nucleo dell'architettura in modo iterativo, si rivolgono i rischi maggiori, si identifica la maggior parte dei requisiti e si formulano stime molto più realistiche e precise

3. **Costruzione (Construction)**

Si sviluppano in modo iterativo gli elementi rimanenti del software, che solitamente sono più facili e a minor rischio, preparando il sistema per la capacità operative iniziali

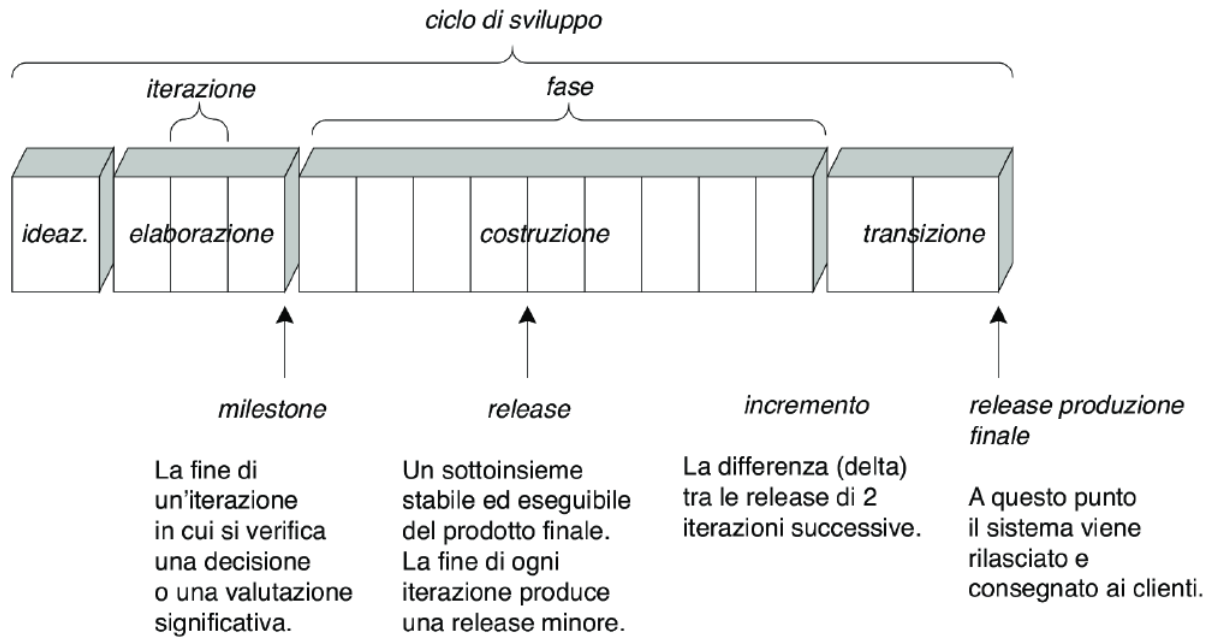
4. **Transizione (Transition)**

È la fase di completamento, che include il beta testing e si conclude con la consegna del prodotto finale ai clienti

2.4 Ciclo di vita

Nello sviluppo UP, il lavoro procede in modo iterativo, incrementale ed evolutivo. Ogni iterazione è considerata come un "mini-progetto" che include una porzione di tutte le attività fondamentali: pianificazione, analisi, progettazione, costruzione, integrazione e test. Al termine di ogni iterazione viene

prodotta una release, ovvero un sottoinsieme stabile ed eseguibile del prodotto finale, che costituisce un passo in avanti (incremento) verso il rilascio definitivo.



2.5 Vantaggi

- **Flessibilità e personalizzazione**

UP è un framework molto flessibile e aperto, che permette di applicare lo scenario "UP Agile" rendendo del tutto opzionali la gran parte dei documenti e modelli e incoraggiano l'integrazione di pratiche da altri metodi agili

- **Riduzione precoce dei rischi**

Il processo affronta le problematiche a maggior rischio e a più alto valore fin dalle primissime iterazioni, costruendo rapidamente un nucleo architettonico coeso

- **Qualità e feedback continui**

Il coinvolgimento costante dell'utente, i test frequenti e i feedback continui a una minore probabilità di fallimento del progetto, a una migliore produttività e a una riduzione dei difetti del software

- **Progresso visibile**

A differenza dei metodi tradizionali, garantisce un progresso visibile fin dall'inizio e aiuta a gestire la complessità del sistema permettendogli di convergere gradualmente verso i requisiti corretti

2.6 Svantaggi

- **Scarsa visibilità per il management**

Il processo risulta meno "visibile" dal punto di vista burocratico.

I manager spesso hanno bisogno di documenti regolari per misurare i progressi, ma sviluppano il software così rapidamente non è conveniente produrre documenti formali per ogni versione

- **Degrado della struttura del software**

Poiché il sistema cresce in modo incrementale con continue aggiunte, la sua struttura tende a degradarsi nel tempo.

Se non si dedica regolarmente tempo e denaro al refactoring per migliorare il codice, incorporare ulteriori modifiche diventerà sempre più difficile e costoso

- **Rischi di cattiva interpretazione**

Pur essendo un processo agile, UP viene spesso frainteso.

Se il team cerca di pianificare tutto in anticipo, definire tutti i requisiti prima di programmare, o pretende di scrivere tonnellate di documenti, il progetto rischia di fallire perché viene invaso da un approccio rigido non adatto alla natura iterativa del framework.

3. Agile

3.1 Cosa è?

Lo sviluppo AGILE è una forma specifica di sviluppo iterativo ed evolutivo progettata per incoraggiare l'**agilità**, ovvero la capacità di rispondere in modo rapido e flessibile ai cambiamenti

A differenza dei processi tradizionali orientati al piano, nei processi agili la pianificazione è incrementale.

3.2 Caratteristiche

- **Iterazioni brevi e a tempo fisso (timeboxed)**
Il lavoro procede in cicli di sviluppo ristretti (spesso da 2 a 4 o 6 settimane) le cui scadenze non possono essere modificate
- **Consegne incrementali**
L'obiettivo è rilasciare frequentemente del software funzionante che fornisca valore tangibile fin da subito
- **Pratiche ingegneristiche disciplinate**
L'agilità è sostenuta da pratiche tecniche specifiche come il Test-Driven Development (TDD) e il continuo refactoring per mantenere il codice pulito, affiancate spesso dalla programmazione a coppie

3.3 Manifesto

La filosofia agile è stata formalizzata nel 2001 da un gruppo di esperti che ha redatto il Manifesto per lo Sviluppo Agile di Software. Questo documento stabilisce quattro priorità di base:

- **Gli individui e le interazioni** contano più che i processi e gli strumenti
- **Il software funzionante** conta più che la documentazione esaustiva
- **La collaborazione con il cliente** conta più che la negoziazione dei contratti
- **Rispondere al cambiamento** conta più che seguire un piano

3.4 Vantaggi

- Risposta flessibile ai cambiamenti
- Software funzionante e progressi visibili
- Feedback continuo e collaborazione
- Riduzione precoce dei rischi

3.5 Svantaggi

- **Scarsa visibilità per i manager**
Il manifesto AGILE preferisce il software funzionante rispetto alla documentazione esaustiva. Sviluppando e modificando il sistema in modo così rapido, non è conveniente produrre lunghi

documenti per ogni versione.

Questo rende il processo "meno visibile" dal punto di vista burocratico per i manager, che spesso dipendono da quei documenti per misurare l'avanzamento

- **Degrado fisiologico della struttura del sistema**

Poichè il sistema non viene progettato interamente in anticipo ma cresce in modo incrementale con aggiunte continue, la sua struttura architettonica tende inesorabilmente a corrompersi nel tempo

- **Dipendenza rigorosa dal refactoring**

Per contrastare il degrado dell'architettura e mantenere il software flessibile, i team agili sono obbligati a investire costantemente tempo e denaro in pratiche ingegneristiche disciplinate come il refactoring del codice. Se questa pratica viene trascurata, incorporare le future modifiche richiederà uno sforzo sempre più grande e costoso.

4. SCRUM

4.1 Cosa è?

Scrum è un metodo agile progettato per sviluppare e rilasciare prodotti software che offrano il più alto valore possibile per i clienti, nel minor tempo possibile.

A differenza di altre metodologie focalizzate sugli aspetti puramente tecnici, Scrum si concentra in modo particolare sull'organizzazione del lavoro e sulla gestione dei progetti, motivo per cui può essere facilmente combinato con altri metodi.

4.2 Caratteristiche

- Approccio Agile
- Focus gestionale
- Ciclo di lavoro (Sprint)
- Allineamento quotidiano
- Suddivisione dei ruoli
- Gestione dei requisiti (Backlog)
- Monitoraggio

4.2.1 Concetti Chiave

- **Incremento potenzialmente rilasciabile**
È il pezzo di software (incremento) fornito al termine di ogni singolo Sprint.
La regola di base impone che questo software sia in uno stato "finito" e teoricamente pronto per la consegna al cliente, senza bisogno di ulteriore lavoro o test per essere integrato nel prodotto finale (anche se, nella pratica, ciò non è sempre facilmente realizzabile).
- **Velocity**
È una stima della quantità di lavoro che il team è in grado di completare all'interno di un singolo Sprint.
Comprendere la propria velocity aiuta la squadra a pianificare meglio le iterazioni future e fornisce un metro di misura per valutare il miglioramento delle prestazioni nel tempo.

4.3 Ruoli principali

- **Scrum Master**
Il responsabile che si assicura che il processo Scrum venga seguito correttamente, guidando il team nel suo utilizzo efficace.
Fa da interfaccia con il resto dell'azienda e ha il compito fondamentale di proteggere il team di sviluppo da interferenze esterne.
Gli sviluppatori agili sottolineano che questa figura non deve essere considerata un project manager tradizionale.
- **Product Owner**
È l'individuo (o un piccolo gruppo) incaricato di identificare i requisiti e le caratteristiche del

prodotto.

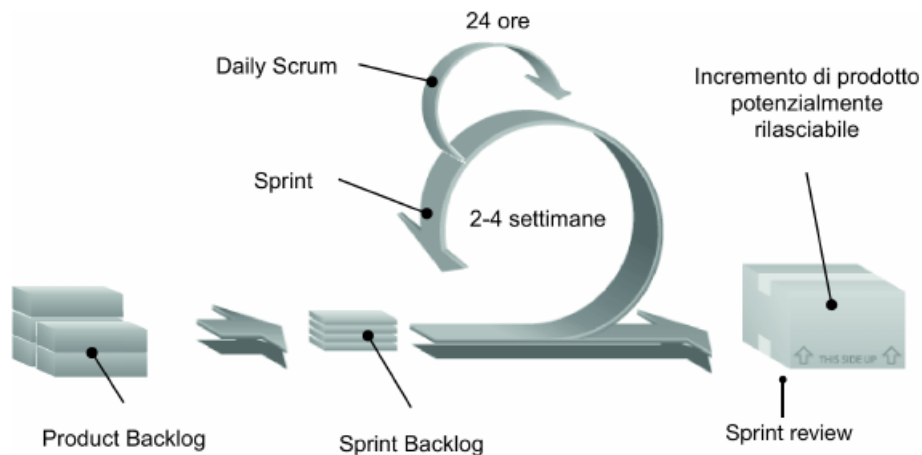
Il suo compito è dettare le priorità di sviluppo e revisionare continuamente la lista delle cose da fare (Product Backlog) per garantire che il progetto soddisfi sempre le esigenze di business critiche. Può essere un cliente, un product manager o un rappresentante degli stakeholder.

- **Team di sviluppo**

È un gruppo auto-organizzato di sviluppatori software, che idealmente non dovrebbe superare le 7 persone.

Sono i responsabili materiali della creazione del software e della stesura dei documenti essenziali del progetto.

4.4 Ciclo di lavoro



- **Sprint**

È l'iterazione base dello sviluppo. Il lavoro viene diviso in Sprint che hanno una durata fissa, solitamente compresa tra le 2 e le 4 settimane

- **Scrum**

È un incontro giornaliero, idealmente faccia a faccia e molto breve, in cui l'intero team si riunisce per esaminare i progressi fatti e definire le priorità del lavoro da svolgere in quella specifica giornata

4.5 Artefatti

- **Product Backlog**

È l'elenco completo di tutti gli elementi "da fare" che il team dovrà affrontare nel corso del progetto. Può contenere requisiti, storie utente, caratteristiche del software o compiti tecnici (come definire l'architettura o scrivere la documentazione).

- **Sprint Backlog**

È il sottoinsieme di compiti e funzionalità estratti dal Product Backlog che il team decide di affrontare e completare nello specifico Sprint in corso.

4.6 Vantaggi

- **Valore rapido per il cliente**

L'intero metodo è strutturato per massimizzare il valore rilasciato al cliente nel più breve lasso di tempo possibile.

- **Altamente compatibile**

Grazie al suo focus organizzativo piuttosto che puramente tecnico, può essere facilmente affiancato e combinato con altri metodi ingegneristici o iterativi.

- **Misurabilità delle performance**

- **Progressi e feedback continui**

4.7 Svantaggi

- **Ideale vs Pratica nel rilascio**

La regola teorica di Scrum prevede che al termine di ogni Sprint si produca un software in uno stato finito, senza bisogno di ulteriore lavoro o test per essere integrato (Incremento potenzialmente rilasciabile);

tuttavia, nella pratica, questo livello di completezza non è sempre facilmente realizzabile.

- **Equivoci sui ruoli**

Per quanto gli esperti Scrum sottolineino che lo ScrumMaster non debba essere considerato un classico "project manager", per il resto dell'azienda può non essere sempre facile cogliere questa differenza, creando confusione a livello organizzativo.

- **Scarsa visibilità burocratica (tipico dell'Agile)**

Come tutti i metodi iterativi veloci, non produce corposi documenti per ogni singola versione rilasciata.

Questo rende l'avanzamento "invisibile" per i manager tradizionali che fanno affidamento sulla documentazione cartacea.

- **Degrado dell'architettura**

Poiché il sistema si sviluppa tramite continui incrementi e accoglie requisiti mutevoli, la sua struttura rischia di corrompersi se non si investono tempo e denaro nel refactoring del codice.

Senza questa pratica, ogni modifica futura diventerà sempre più lenta e costosa.

5. Pattern GRASP

5.1 Definizione

I pattern GRASP (General Responsibility Assignment Software Pattern) sono un insieme di principi fondamentali e linee guida utilizzati nella **progettazione del software orientata agli oggetti** per supportare il progettista nell'assegnazione delle responsabilità a classi e oggetti.

5.1.1 Approccio RDD

Più che schemi di codice preconfezionati, i GRASP deniscano un approccio metodologico noto come **Responsability-Driven Design (Progettazione Guidata alla responsabilità)** formalizzando sotto forma di pattern le regole e le soluzioni idiomatiche applicate.

5.1.2 Principi fondamentali

- **Focus sulle Responsabilità:**

Nella progettazione a oggetti, i requisiti espressi nei casi d'uso vengono soddisfatti traducendoli in responsabilità per le classi del software.

Tali responsabilità si dividono in 2 macro-categorie:

- **Responsabilità di FARE (Doing):** Come eseguire un'azione di business, creare un altro oggetto o coordinare attività.
- **Responsabilità di CONOSCERE (Knowing):** Come la conoscenza dei propri dati privati incapsulati, delle relazioni o di elementi calcolabili.

- **Applicazione Dinamica:**

Lo strumento principale in cui si applicano e si concretizzano i pattern GRASP è il disegno dei **diagrammi di interazione UML**.

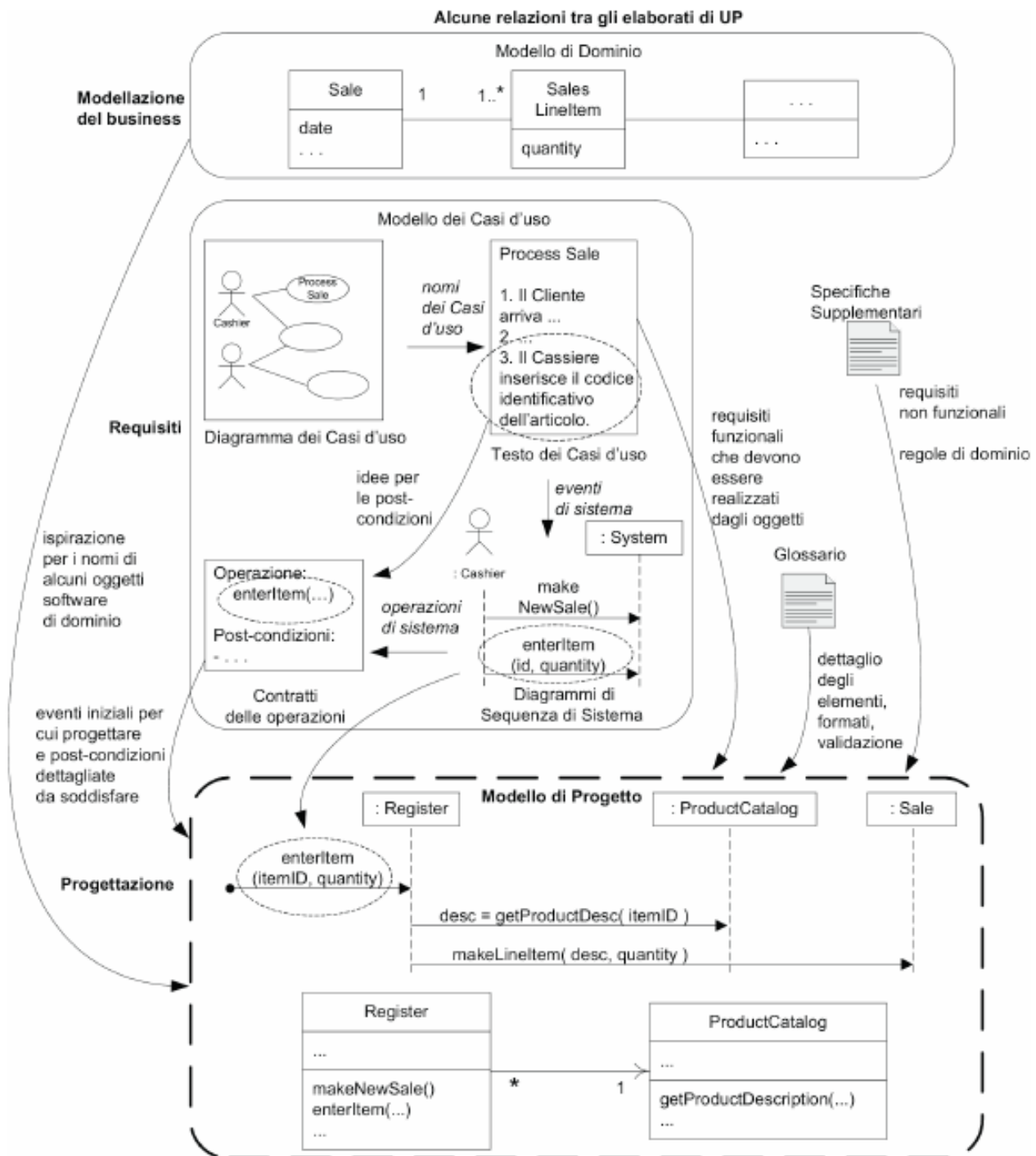
È proprio nel momento in cui si definiscono i messaggi scambiati tra gli oggetti che il progettista sceglie a quale entità assegnare una specifica funzione.

- **Obiettivo Qualificativo:**

L'applicazione metodica di questi pattern mira a massimizzare metriche software essenziali, riducendo l'impatto dei cambiamenti futuri e migliorando la manutenibilità.

I due capisaldi di questo obiettivo sono il raggiungimento di un **accoppiamento basso**(Low Coupling) e di una **coesione alta**(High Cohesion)

5.1.3 Relazione tra gli elaborati di UP



5.2 Rappresentazione di un Pattern GRASP

Nome del pattern: Information Expert

Problema: Qual è un principio di base con cui assegnare responsabilità agli oggetti?

Soluzione: Assegna una responsabilità alla classe che ha le informazioni necessarie per soddisfarla.

6. Pattern GRASP di base

6.1 Creator

Nome:	Creator (Creatore)
Problema:	Chi crea un oggetto A?
Soluzione	Assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera (più sono vere, meglio è):
(può essere	• B “contiene” o aggrega con una composizione oggetti di tipo A. • B registra A. • B utilizza strettamente A. • B possiede i dati per l'inizializzazione di A.
vista come un consiglio):	

6.1.1 Problema

In un softwaread oggetti, la creazione di nuovi oggetti è un'operazione pervasiva.

Tuttavia, se una classe crea direttamente un'altra classe,diventa fortemente accoppiata a quest'ultima. Se la classe creata **cambia o viene sostituita**, anche la classe che la crea potrebbe subire modifiche.

Il problema è quindi: Come assegnare la responsabilità di creazione in modo da non aumentare l'accoppiamento complessivo al sistema?

6.1.2 Soluzione

Il pattern Creator stabilisce che una classe B dovrebbe ricevere la responsabilità di creare un'istanza della classe A se si verifica almeno una (e preferibilmente più di una) delle seguenti condizioni:

1. **Aggregazione/Composizione:** B contiene, aggrega o racchiude gli oggetti di tipo A
2. **Registrazione:** B registra o tiene traccia delle istanze di A
3. **Uso Stretto:** B utilizza strettamente ed estesamente gli oggetti di tipo A
4. **inizializzazione:** B possiede i dati di inizializzazione che devono essere passati ad A al momento della sua creazione

6.1.3 Benefici

- Basso Accoppiamento
- Incapsulamento

6.1.4 Controindicazioni

Non sempre il pattern Creator è la scelta migliore.

Esistono situazioni in cui l'applicazione rigida di questo principio crea problemi:

- **Logica di creazione complessa:**

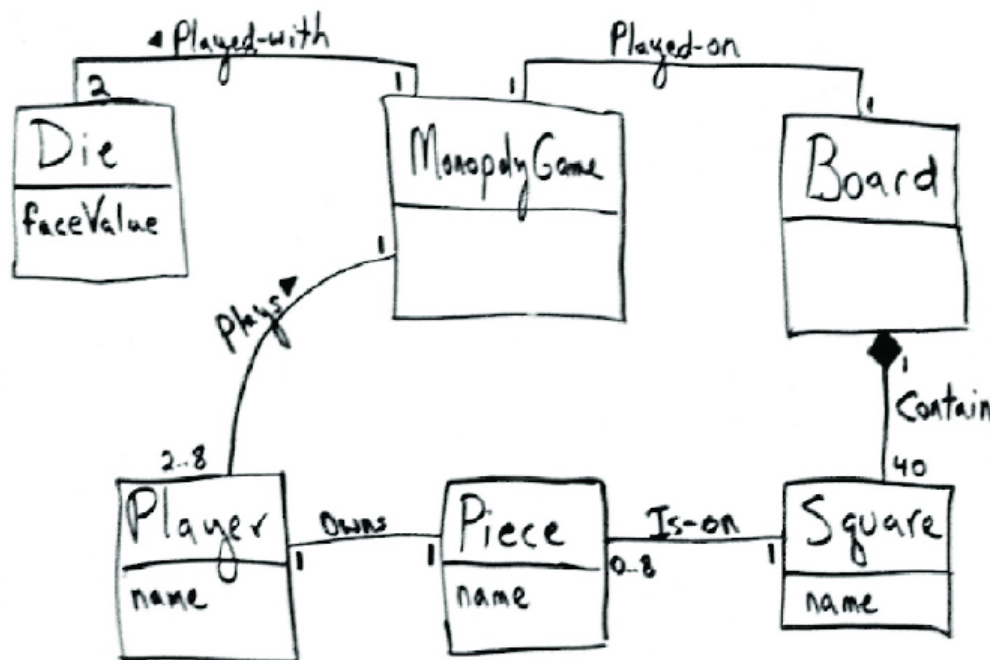
Se la creazione richiede passaggi condizionali intricati, la scelta dinamica di sottoclassi o l'interfaciamento con librerie esterne, assegnare la creazione alla classe aggregatrice potrebbe distruggere la coesione

- **Riuso del processo di creazione:** Se lo stesso identico processo di creazione serve a molte classi diverse

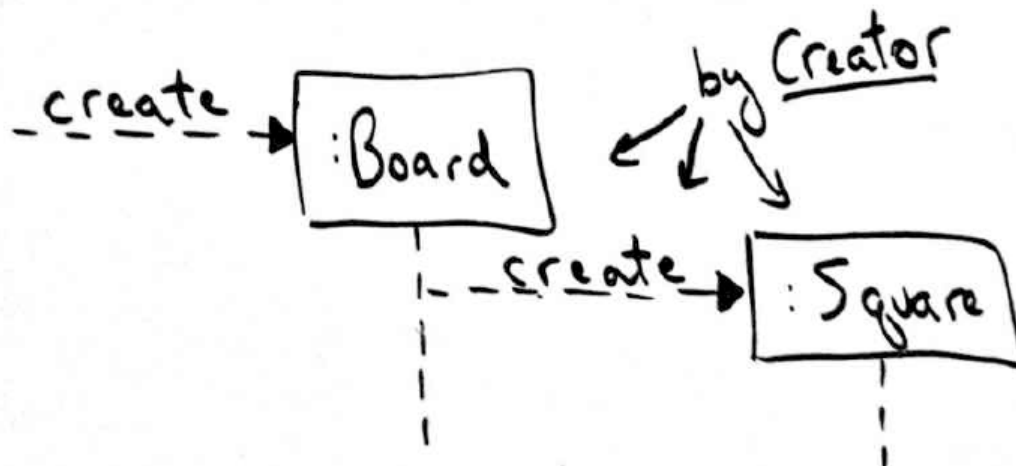
6.1.5 Flusso di lavoro logico

1. Leggi il contratto dell'operazione di sistema (che ti dice cosa deve essere creato)
2. Nel diagramma di sequenza si delinea chi crea cosa
3. Si riporta la funzione e la freccia di dipendenza nel diagramma delle classi

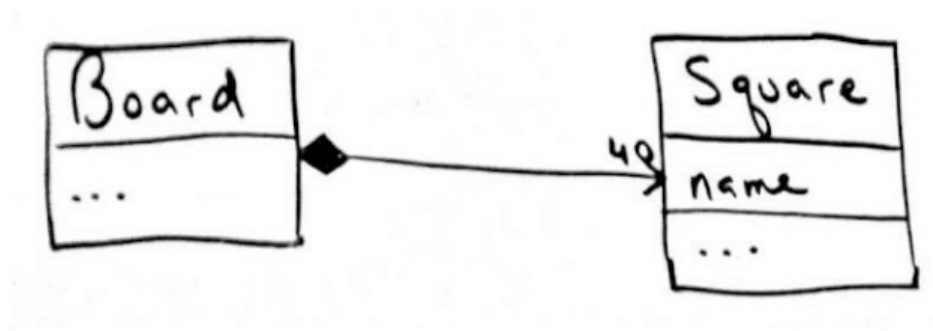
6.1.6 Modello di dominio



6.1.7 Modello di sequenza



6.1.8 Diagrammi delle classi di progetto



6.2 Information Expert

Nome del pattern: Information Expert (Esperto dell'Informazione)

Problema: Qual è un principio di base fondamentale con cui assegnare le responsabilità agli oggetti?

Soluzione: Assegna una responsabilità alla classe che possiede le informazioni necessarie per soddisfarla.

6.2.1 Problema

Nello sviluppo software, quando si riscrive un nuovo caso d'uso, ci si trova davanti a decine di frammenti di logica di business.

Se queste responsabilità vengono assegnate a caso, si rischia di scivolare nell'approccio procedurale, dove le classi di dominio diventano dei semplici contenitori anemidici di dati, mentre tutta la logica operativa viene accentrata in un unico enorme blocco. Questo porta a un codice di bassa qualità

Information Expert risolve questo caso fornendo un criterio oggettivo: esamina i dati e mette l'azione dove risiedono i dati necessari a compierla

6.2.2 Soluzione

La direttiva è semplice: guarda quale classe possiede le informazioni richieste per completare il compito e assegna il metodo a quella classe.

Le "informazioni" necessarie possono essere:

- Attributi privati della classe stessa
- Oggetti strettamente correlati ad essa
- Informazioni che la classe può calcolare o derivare interrogando i suoi componenti

Se le informazioni sono distribuite su più classi, l'information Expert diventa un concetto incrementale: si crea una catena di deleghe in cui ogni classe fa la sua parte basandosi su ciò che conosce direttamente.

6.2.3 Benefici

- Mantenimento dell'incapsulamento
- Alta Coesione
- Basso Accoppiamento

- Facilità di manutenzione

6.2.4 Controindizioni

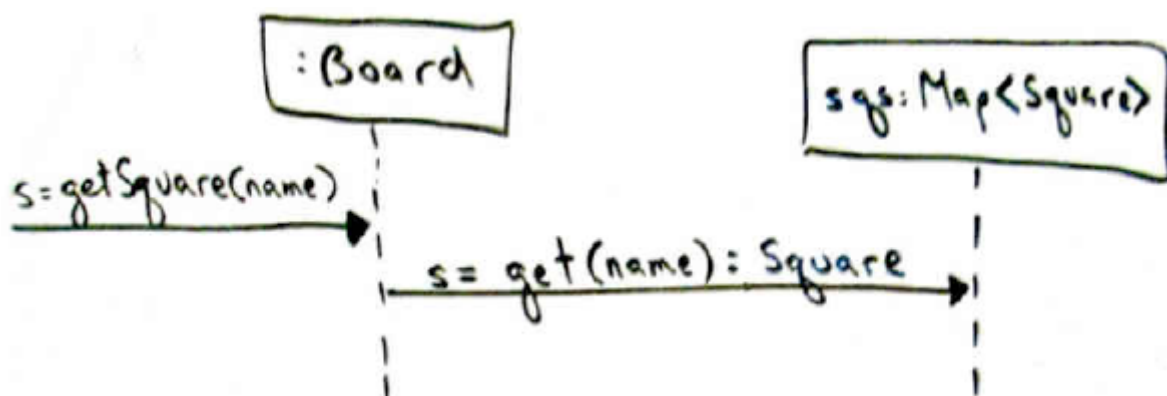
- Problemi di persistenza
- Logiche di visualizzazione (UI):

Chi conosce i dati dell'utente? La classe User. Ma non dobbiamo permettere a User di stampare se stessa in HTML o di aprire una finestra pop-up grafica. La responsabilità grafica va separata nettamente.

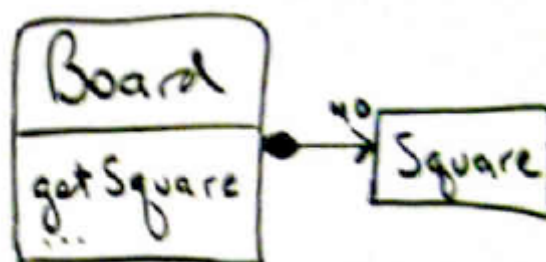
6.2.5 Flusso di lavoro

1. **Identifica il requisito:** Leggi i contratti delle operazioni o i casi d'uso
2. **Cerca le informazioni:**
Guarda il modello di dominio e individua quali classi possiedono gli attributi necessari a quel calcolo
3. **Inizia dall'alto:** Trova la classe che fa da aggregatore principale e assegna il metodo di alto livello
4. **Applica la delega:**
Se la classe principale non ha tutti i dettagli, scendi di un livello nell'albero delle associazioni e assegna sotto-responsabilità alle classi componenti
5. **Disegna l'interazione:**
Convalida questa scelta tracciando i messaggi nel diagramma di sequenza.
Se le frecce fluiscono in modo lineare seguendo la struttura dei dati, l'applicazione dell'information Expert è corretta

6.2.6 Diagramma di sequenza



6.2.7 Diagramma delle classi



6.3 Low Coupling

Nome del pattern: **Low Coupling** (Accoppiamento Basso)

Problema: Come ridurre l'impatto dei cambiamenti, supportare un alto riuso e facilitare la manutenzione?

Soluzione: Assegna le responsabilità in modo tale che l'accoppiamento (le dipendenze tra le classi) rimanga il più basso possibile.

6.3.1 Problema

L'accoppiamento è la misura di quanto fortemente un elemento è connesso a, conosce o dipende da altri elementi.

Se un sistema ha un alto accoppiamento, le classi sono strettamente interconnesse tra loro in una rete ingarbugliata.

Questo approccio comporta 3 gravi problemi:

- **Effetto Dominio:** Una modifica apparentemente innocua in una classe rompe il funzionamento di altre 10 classi sparse nel codice
- **Impossibilità di riuso**
- **Difficoltà di comprensione**

6.3.2 Soluzione

Il pattern suggerisce di valutare costantemente le alternative di progettazione e di scegliere sempre la soluzione che minimizza i legami di dipendenza.

In un linguaggio orientato agli oggetti, la classe A è accoppiata alla classe B se:

- A ha un attributo che fa riferimento a un istanza di B
- A invoca i metodi di un oggetto B
- A ha un metodo che riceve come argomento o restituisce un oggetto B
- A è una sottoclasse di B

6.3.3 Benefici

- **Isolamento delle modifiche**
- **Elevato riuso**
- **Testabilità semplificata**

6.3.4 Controindicazioni

- **Metodi ad-hoc gonfiati**
- **Infrastrutture e standard stabili**

6.3.5 Flusso di lavoro

1. Abbozza le alternative:

Quando devi implementare un messaggio di sistema pensa ad almeno due modi diversi in cui gli oggetti potrebbero cooperare per raggiungere il risultato

2. Conta le linee di dipendenza:

Guarda le frecce del tuo diagrama di interazione.

Quanti oggetti diversi deve conoscere la classe di partenza per portare a termine il compito?

3. Valuta la stabilità: Se un accoppiamento è inevitabile, cerca di indirizzarlo verso l'oggetto più stabile

4. Trova il compromesso con la coersione:

Assicurati che l'alternativa scelta per ridurre l'accoppiamento non distrugga l'alta coesione della classe ricevente

6.4 High Cohesion

Nome del pattern: **High Cohesion** (Coesione Alta)

Problema: Come mantenere gli oggetti focalizzati, comprensibili e gestibili, supportando al contempo un basso accoppiamento?

Soluzione: Assegna le responsabilità in modo che le operazioni interne di una classe siano strettamente correlate dal punto di vista funzionale.

6.4.1 Problema

Nello sviluppo di sistemi complessi, c'è la forte tentazione di inserire nuove funzioni all'interno delle poche classi già esistenti e note, creando involontariamente delle "classi mostro".

Se una classe ha una **bassa coesione**, significa che sta svolgendo troppi compiti eterogenei che non c'entrano nulla l'uno con l'altro.

Questo scenario comporta gravi problemi:

- **Fragilità e difficoltà di lettura**
- **Scarsa riusabilità**
- **Saturazione da cambiamento:** La classe sarà costretta a cambiare ogni volta che cambia uno qualsiasi dei cinque requisiti di business differenti di cui si fa carico

6.4.2 Soluzione

Il pattern suggerisce di **delegare** i compiti specialistici e mantenere ogni classe **specializzata** in un unico blocco logico di funzionalità. Una classe ad alta coesione fa "una sola cosa fatta bene".

6.4.3 Benefici

- **Facilità di comprensione**
- **Manutenzione localizzata**
- **Sviluppo in parallelo**

6.4.4 Controindicazioni

L'eccesso opposto della bassa coesione è la frammentazione esasperata, se si applica il principio in modo paranoico, si rischia di creare centinaia di microscopiche classi formate da un solo metodo e tre righe di codice ciascuna.

Questo approccio non fa altro che nascondere la complessità del sistema dentro un labirinto di collegamenti e chiamate, peggiorando drasticamente l'accoppiamento del software e rendendo il debugging un incubo.

Una coesione moderatamente alta è il target ideale.

6.4.5 Flusso di lavoro

Per garantire un'alta coesione durante le sessioni di progettazione nei diagrammi di sequenza e delle classi:

1. **Definisci l'identità della classe**
2. **Monitora i metodi aggiunti**
3. **Ispezione le variabili di istanza**
4. **Applica la delega attiva**

6.5 Controller

Nome del pattern:	Controller (Controllore)
Problema:	Qual è il primo oggetto oltre lo strato della UI che riceve e coordina l'esecuzione di un'operazione di sistema?
Soluzione:	Assegna questa responsabilità a un oggetto che rappresenti l'intero sistema, un dispositivo logico, o lo scenario del caso d'uso corrente.

6.5.1 Problema

Quando un utente interagisce con un'interfaccia grafica, l'interfaccia cattura un evento software.

Se il codice che esegue l'operazione di business viene scritto direttamente dentro il codice del pulsante grafico, si commette un grave errore di progettazione.

Questo approccio genera 2 problemi principali:

- **Dipendenza dalla tecnologia visiva**
- **Perdita di controllo e coesione**

Il controller risolve questo problema fungendo da intermediario e coordinatore

6.5.2 Soluzione

Il pattern stabilisce che la UI deve limitarsi a catturare il clic dell'utente e delegarlo direttamente a un oggetto specifica che sta ne background, chiamato appunto controller.

Il GRASP suggerisce due modi principali per scegliere o creare un oggetto controller:

- **Facade Controller**
Si assegna la responsabilità a un oggetto che rappresenta il sistema complessivo, il software nel suo insieme o il dispositivo fisico in uso
- **Use-Case Controller**
Si crea un oggetto che rappresenta l'intero scenario o la sequenza di azioni di un singolo caso d'uso

6.5.3 Benefici

- Disaccoppiamento della UI
- Riutilizzo dei canali di input
- Facilità di test

6.5.4 Controindicazioni

Il rischio più comune quando si implementa questo pattern è la creazione di un Controllore Gonfio.

6.5.5 Flusso di lavoro

1. **Analizza i diagrammi di sequenza di sistema:** Identifica quali sono messaggi che l'attore invia al sistema
2. **Scegli il tipo di controller**
3. **Piazza il blocco nel diagramma di interazione:** Nei diagrammi di sequenza dettagliati, inserisci la linea di vita della UI alla sinistra e, come secondo oggetto subito a destra, inserisci l'oggetto Controller
4. **Disegna la delega:** Fai partire la freccia dalla UI al Controller. Subito dopo, fai partire le frecce dal Controller verso le vere classi del Modello di Dominio applicando l'Information Expert.

7. Pattern GRASP avanzati

7.1 Pure Fabrication

Nome del pattern: Pure Fabrication

Problema: Come assegnare una responsabilità senza violare alta coesione e basso accoppiamento quando pattern come l'Information Expert falliscono?

Soluzione: Creare una classe artificiale (che non esiste nel dominio) e assegnarle un set di responsabilità fortemente coeso

7.1.1 Problema

Sorge quando si cerca di assegnare una responsabilità utilizzando i pattern standard, ma facendolo si violerebbero obiettivi cruciali come mantenere un'alta coesione e un basso accoppiamento

7.1.2 Soluzione

Consiste nell'assegnare la responsabilità a una classe "artificiale" o di convenienza.

Questa classe non rappresenta un concetto reale nel dominio del problema, ma viene inventata appositamente dal progettista per sostenere alta coesione, accoppiamento basso e riuso

7.1.3 Benefici

- Sostiene l'alta coesione
- Aumenta significativamente la potenzialità di riuso del codice

7.1.4 Controindicazioni

- Un utilizzo eccessivo può portare a una "decomposizione del codice"
- Rischio di creare un eccesso di oggetti puramente comportamentali che non possiedono le informazioni richieste per soddisfare i propri compiti

7.1.5 Flusso di lavoro

1. Identificare che l'assegnazione di una responsabilità violerebbe l'alta coesione o il basso accoppiamento di una classe esistente
2. Inventare una classe "artificiale" che non fa parte del dominio del problema
3. Assegnare a questa nuova classe l'insieme di responsabilità altamente coese
4. Progettare un'opportuna interazione coordinata dalla Pure Fabrication assicurandosi di passarle le informazioni necessarie per operare

7.2 Polymorphism

Nome del pattern:	Polymorphism
--------------------------	---------------------

Problema:	Come gestire alternative basate sul tipo e creare componenti inseribili?
------------------	--

Soluzione:	Assegnare la responsabilità del comportamento ai tipi per i quali varia, usando operazioni polimorfe
-------------------	--

7.2.1 Problema

Come gestire alternative e comportamenti correlati che sono basate sul tipo? Come creare? comportamento software "inseribili"?

7.2.2 Soluzione

Quando le alternative variano con il tipo, si assegna la responsabilità del comportamento direttamente ai tipi per i quali il comportamento varia, utilizzando operazioni polimorfe

7.2.3 Benefici

- Le estensioni richieste per gestire nuove variazioni sono facili da aggiungere
- Si possono introdurre nuove implementazioni senza influire minimamente sui client che le utilizzano

7.2.4 Controindicazioni

- Progettare interfacce e polimorfismi per una "adattabilità" speculativa porta spesso alla progettazione di variazioni improbabili che, nella realtà, non si verificheranno mai

7.2.5 Flusso di lavoro

1. Rilevare la presenza di un comportamento che necessita di alternative differenti basate sul tipo
2. Definire un'astrazione comune con un'operazione polimorfa
3. Assegnare la responsabilità di implementare lo specifico comportamento alle varie sottoclassi o tipi concreti

7.3 Indirection

Nome del pattern: Indirection

Problema: Come disaccoppiare gli oggetti ed evitare l'accoppiamento diretto tra elementi?

Soluzione: Assegnare la responsabilità a un oggetto intermediario che faccia da mediatore tra i componenti

7.3.1 Problema

Dove assegnare una responsabilità per evitare l'accoppiamento diretto tra più elementi?

Come disaccoppiare gli oggetti in modo da sostenere un alto riuso e un accoppiamento basso?

7.3.2 Soluzione

Si assegna la responsabilità a un oggetto intermediario che ha il suo compito di mediare tra altri componenti o servizi, evitando così che vi sia un accoppiamento diretto tra di essi

7.3.3 Benefici

- Crea una utile indirizione tra le diversi componenti
- Garantisce un accoppiamento notevolmente più bassp tra i componenti del sistema

7.3.4 Controindicazioni

- L'aggiunta di intermediari aumenta fisiologicamente il numero di classi e la complessità strutturale del codice

7.3.5 Flusso di lavoro

1. Individuare due o più componenti o servizi che rischiano di essere accoppiati direttamente in modo rigido
2. Creare un nuovo oggetto con la funzione di intermediario (come un Adapter)
3. Instradare le chiamate del client verso l'intermediario, in modo che sia quest'ultimo a comunicare con il servizio di destinazione disaccoppiando le due parti

7.4 Protected Variations

Nome del pattern: Protected Variations

Problema: Come evitare che variazioni o instabilità si riversino su altri elementi del sistema?

Soluzione: Identificare i punti di variazione e assegnare le responsabilità per creare un'interfacciastabile attorno ad essi

7.4.1 Problema

Come progettare oggetti, sottoinsieme e sistemi in modo tale che le variazioni o l'instabilità di questi elementi non si riversino in modo distruttivo su altri elementi

7.4.2 Soluzione

Si identificano i punti in cui sono previste delle variazioni, poi si assegnano le responsabilità in modo da creare una "interfaccia" stabile e inossidabile attorno a questi punti critici

7.4.3 Benefici

- Le estensioni richieste per gestire nuove variazioni sono facili da aggiungere
- È possibile introdurre nuove implementazioni senza influire sui client
- Favorisce un basso accoppiamento
- È possibile ridurre drasticamente l'impatto o il costo dei cambiamenti futuri

7.4.4 Controindicazioni

- È un'operazione inutile e costosa utilizzare questo pattern per proteggersi da variazioni che non si verificheranno mai
- In alcune circostanze, risulta più conveniente e meno complesso riprogettare il software solo quando la variazione si presenta, piuttosto che tentare di realizzare preventivamente scudi contro ogni potenziale variazione immaginabile

7.4.5 Flusso di lavoro

1. Analizzare il sistema per identificare con precisione i punti di instabilità o le aree in cui si prevedono variazioni future
2. Assegnare le responsabilità in modo da isolare quegli specifici punti dal resto del sistema
3. "Avvolgere" l'elemento instabile fornendo una stabile interfaccia di comunicazione, bloccando così la propagazione a cascata dei futuri cambiamenti

8. Design Pattern GoF

8.1 Cosa sono?

I design pattern GoF (Gang of Four) sono soluzioni progettuali comuni a problemi di progettazione ricorrenti nello sviluppo di software a oggetti.

In termini pratici, rappresentano un insieme specifico di oggetti e classi a cui vengono date delle responsabilità e che collaborano tra loro per risolvere un determinato problema.

8.2 Confronto con i pattern GRASP

I pattern GRASP e i pattern GoF sono due concetti complementari ma con un livello di astrazione diverso.

8.2.1 Principi contro Strutture

- I **pattern GRASP** sono principi di base che guidano il progettista nell'assegnare le responsabilità.
- I **pattern GoF** sono invece soluzioni strutturate specifiche a problemi ricorrenti

8.3 Adapter

Nome del pattern: Adapter

Problema: Come gestire interfacce incompatibili, o fornire un'interfaccia stabile a componenti simili ma con interfacce diverse?

Soluzione: Converti l'interfaccia originale di un componente in un'altra interfaccia, attraverso un oggetto adattatore intermedio

8.3.1 Problema

Si verifica quando un oggetto client deve fruire di servizi da un oggetto server, ma l'interfaccia offerta dal server è incompatibile con quella attesa dal client e non può essere modificata

8.3.2 Soluzione

Interporre un oggetto intermedio che traduce le richieste del client nel formato specifico richiesto dal componente esterno

8.3.3 Benefici

- Sostiene la Protected Variations rispetto ai cambiamenti o alle differenze delle interfacce esterne
- Aggiugne un utile livello di indirection alle API variabili

8.3.4 Controindicazioni

- Non specificate nei documenti forniti

8.3.5 Implementazione

1. Definire un'interfaccia generica (il Target) che il client si aspetta di utilizzare
2. Crea una classe specifica (Adapter) che implementi questa interfaccia Target
3. All'interno del metodo implementato dalla classe Adapter, richiamare la logica invocando il metodo specifico della classe esterna Adaptee incompatibile

8.3.6 Dettagli

In termini di pattern GRASP, l'Adapter è una specie di Pure Fabrication e Indirection che utilizza il Polymorphism

8.4 Factory

Nome del pattern: **Factory**

Problema: Chi deve essere responsabile della creazione di oggetti quando ci sono delle considerazioni speciali, come una logica di creazione complessa, quando si desidera separare le responsabilità per una coesione migliore?

Soluzione: Crea un oggetto Pure Fabrication chiamato Factory che gestisce la creazione

8.4.1 Problema

Assegnare la responsabilità di creare oggetti complessi (come gli adattatori verso l'esterno) direttamente agli oggetti di dominio diminuisce la coesione, mescola la logica applicativa pura con logiche di istanziazione e viola il principio della separazione degli interessi

8.4.2 Soluzione

Delegare la logica di creazione a un oggetto di supporto altamente coeso (Pure Fabrication) chiamato Factory

8.4.3 Benefici

- Separa le responsabilità delle creazioni complesse collocandole in oggetti di supporto coesi
- Nasconde al resto del sistema logiche di creazione potenzialmente complesse
- Consente l'introduzione di strategie avanzate per la gestione delle risorse, come il caching o il riciclaggio degli oggetti

8.4.4 Implementazione

1. Creare una classe artificiale dedicata al ruolo di Factory
2. Dichiarare al suo interno i metodi per la creazione, facendo attenzione a restituire sempre oggetti tipizzati come interfacce anzichè come classi concrete, in modo che il client non conosca l'implementazione esatta
3. All'interno del metodo, implementare l'istanziamento.
Si può utilizzare un approccio basato sulla riflessione o "data-driven": leggere da un file di configurazione il nome esatto della classe da usare e istanziarla dinamicamente

8.4.5 Dettagli

L'approccio data-driven garantisce Protected Variations riguardo la scelta degli adattatori da usare.

Tipicamente, l'oggetto Factory viene reso accessibile tramite il pattern Singleton.

8.5 Singleton

Nome del pattern: Singleton

Problema: È consentita (o richiesta) esattamente una sola istanza di una classe. Gli altri oggetti hanno bisogno di un punto di accesso globale e singolo a questo oggetto.

Soluzione: Definisci un metodo statico (di classe) della classe che restituisce l'oggetto singleton

8.5.1 Problema

Sorge la necessità di determinare l'esistenza di una sola istanza di una determinata classe, e che tale istanza sia accessibile globalmente da più punti del codice

8.5.2 Soluzione

Fornire alla classe un metodo statico responsabile di creare e restituire sempre l'unica istanza consentita

8.5.3 Benefici

- Definisce un punto di accesso globale unificato a un singolo oggetto
- Consente l'inizializzazione "pigra" dell'oggetto solo al primo effettivo utilizzo
- È una soluzione architetturealmente preferibile rispetto all'uso di mere variabili globali

8.5.4 Controindicazioni

- Definendo oggetti con visibilità globale, un suo abuso come conseguenza un marcato aumento dell'accoppiamento complessivo del sistema
- Può risultare inefficace in applicazioni distribuite, poichè ciascuna VM potrebbe istanziare una propria diversa versione del Singleton

8.5.5 Implementazione

1. All'interno della classe, dichiara un attributo statico privato deputato a contenere l'istanza
2. Creare un costruttore privato per impedire l'utilizzo di `new` da parti di classe esterne
3. Fornire un metodo pubblico e statico.

Tale metodo verificherà se l'istanza privata è nulla:

- In caso positivo la creerà
- Altrimenti, si limiterà a restituirla

8.5.6 Dettagli

ei diagrammi UML, il pattern Singleton può essere segnalato inserendo opzionalmente il numero 1 nell'angolo in alto a destra della classe

8.6 Strategy

Nome del pattern: Strategy

Problema: Come progettare per gestire un insieme di algoritmi o politiche variabili ma correlate?
Come progettare per consentire di modificare questi algoritmi?

Soluzione: Definisci ciasun algoritmo/politica/strategia in una classe separata, con un'interfaccia comune

8.6.1 Problema

Si necessita di gestire logiche complesse soggette a costanti variazioni nel tempo

8.6.2 Soluzione

Raggruppare gli algoritmi correlati definendo ciascuno di essi all'interno della proprio classe separata e facendogli implementare un'interfaccia unificata

8.6.3 Benefici

- Permette di avere algoritmi "inseribili" senza modificare il client
- Offre solidissime Protected Variations in merito al variare delle politiche di calcolo

8.6.4 Implementazione

1. Definire un'interfaccia per la politica che dichiara il metodo di calcolo atteso
2. Sviluppare classi concrete autonome per singola politica dotate delle specifiche formule di calcolo all'interno del metodo
3. L'oggetto client "contesto" deve dichiarare un attributo in grado di mantenere il riferimento all'interfaccia strategica in uso
4. Il client delegherà l'operazione chiamando polimorficamente il metodo della strategia a cui passerà i dati necessari o se stesso

8.6.5 Dettagli

Si basa fundamentalmente sul Polymorphism. Frequentemente le strategie vengono generate ricorrendo all'utilizzo di oggetti Factory.

8.7 Composite

Nome del pattern: Composite

Problema: Come trattare un gruppo o una struttura composta di oggetti dello stesso tipo nello stesso modo di un oggetto non composto?

Soluzione: Definisci le classi per gli oggetti composti e atomici in modo che implementino la stessa interfaccia

8.7.1 Problema

Sorge quando occorre gestire situazioni che sono simultanee, multiple e conflittuali.

Come fare affinché il client chiami questi componenti composti credendo di interfacciarsi con uno singolo?

8.7.2 Soluzione

Progettare le classi composte e le classi foglia atomiche cosicchè entrambe onorino la medesima interfaccia

8.7.3 Benefici

- Attraverso il Polymorphism, fornisce preziose Protected Variations lato client: il codice chiamante non subisce ripercussioni se sta trattando singole regole atomiche o ampi alberi composti

8.7.4 Implementazione

1. Avere un interfaccia comune che rappresenti l'operazione
2. Definire una superclasse composta astratta
3. Fornire alla classe composta una lista intera in cui immagazzinare i propri elementi "figli"
4. Aggiungere metodi per manipolare la lista
5. Nelle sottoclassi concrete del Composite, codificare il metodo operativo affinché utilizzi un ciclo iterativo (loop) per interrogare tutte le strategie registrate nella lista, delegare a esse il calcolo e alla fine sintetizzarne il risultato

8.7.5 Dettagli

È utilizzato comunemente in congiunzione con i pattern Strategy e Command

8.8 Facade

Nome del pattern: Facade

Problema: È richiesta un'interfaccia comune e unificata per un insieme disparato di implementazioni.
Accedervi crea accoppiamento indesiderato o espone ai mutamenti del sottosistema

Soluzione: Definisci un punto di contatto singolo con il sottosistema, ovvero un oggetto facade che copre il sottosistema ed è responsabile della collaborazione

8.8.1 Problema

Interfacciarsi direttamente con un sottosistema popolato da moltissime classi disparate comporta un eccessivo e nocivo accoppiamento per l'oggetto chiamante, specialmente se l'implementazione di quel sottosistema è destinata a cambiare

8.8.2 Soluzione

Creare un oggetto "front end" che rappresenti l'unico e il solo punto di entrata unificato a copertura dall'intero sottosistema

8.8.3 Benefici

- Aiuta a sostenere il basso accoppiamento aggiungendo un fondamentale oggetto intermedio
- Offre potenti Protected Variations isolando il resto del progetto dalle variazioni interne dell'implementazione del sottosistema

8.8.4 Implementazione

1. Includere all'interno del package del dominio/sottosistema una classe pubblica che agisca da interfaccia d'accesso
2. Definire nella classe Facade i metodi per esaudire le richieste provenienti del client
3. Programmare il corpo dei metodi in modo tale che sia il Facade a instradare, segretamente, la logica verso le implementazioni interne delle altre classi del sottosistema

8.8.5 Dettagli

Il tipico facade controller del GRASP è esattamente un tipo di Facade.

Normalmente il Facade è progettato per essere acceduto attraverso il pattern Singleton.

8.9 Observer (Publish-Subscribe)

Nome del pattern: Observer (Publish-Subscribe)

Problema: Diversi subscriber vogliono reagire in un loro modo proprio ai cambiamenti di un publisher.
Il publisher vuole mantenere un accoppiamento basso

Soluzione:

1. Definisci un'interfaccia subscriber o listener.
2. Gli oggetti la implementano.
3. Il publisher li registra dinamicamente e li avvisa degli eventi

8.9.1 Problema

Oggetti eterogenei necessitano di aggiornarsi se muta lo stato in un oggetto sorgente.

Tuttavia, per la qualità architetturale, il publisher non deve conoscere direttamente le classi specifiche di chi riceverà l'aggiornamento per evitare un rigido accoppiamento

8.9.2 Soluzione

Creare un meccanismo basato su una specifica interfaccia che i subscriber devono implementare.

Il publisher mantiene solo una lista generica di iscritti da notificare in blocco quando un evento si manifesta

8.9.3 Benefici

- Assicura un modo per far comunicare gli oggetti con un accoppiamento estremamente debole
- Basandosi sul polimorfismo, favorisce le Protected Variations tutelando l'editore dal dover sapere chi siano le implementazioni che ascoltano il suo evento.

8.9.4 Implementazione

1. Progettare un'interfaccia d'ascolto per i subscriber che contenga il metodo dell'evento
2. Fare implementare questa interfaccia alle classi interessate a farsi notificare
3. Nel Publisher, creare un contenitore per raccogliere le iscrizioni, come una lista o un array di listener
4. Munire il Publisher dei metodi per gestire gli abbonamenti dinamicamente
5. Quando l'evento ha luogo nel Publisher, richiamare un metodo interno (publishPropertyEvent()) che itera (loop) su tutti i listener registrati, chiamando polimorficamente su ognuno il metodo della loro interfaccia e fornendogli i dati.

8.9.5 Dettagli

Oltre a essere il perno della comunicazione tra logica (Modello) e Interfaccia Utente (UI), è utilissimo per processi in background (come Beeper, Timer e WatchDog)

9. Visibilità

9.1 Definizione

È la capacità di un oggetto di "**vedere**" o di **possedere un riferimento** a un altro oggetto.

Questa condizione è requisito strutturale essenziale: affinché un oggetto mittente possa inviare un messaggio a un oggetto destinatario, il destinatario deve rientrare nella portata del mittente, ovvero essergli visibile.

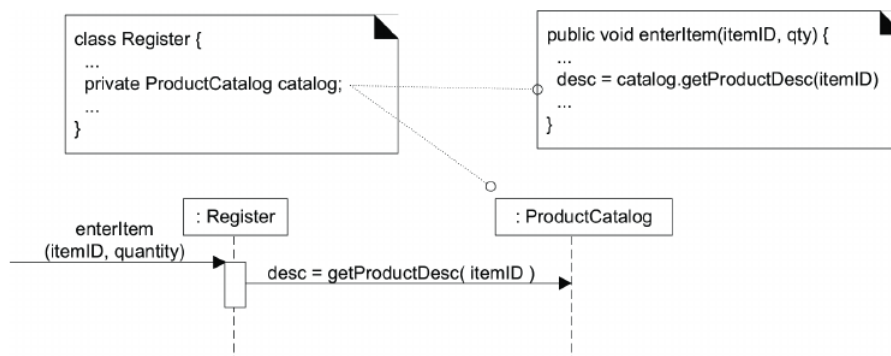
9.2 Tipi di visibilità

La visibilità da un oggetto A a un oggetto B può essere ottenuta in 4 modi comuni.

9.2.1 Visibilità per attributo

Si verifica quando l'oggetto B è dichiarato come un attributo all'interno della classe dell'oggetto A.

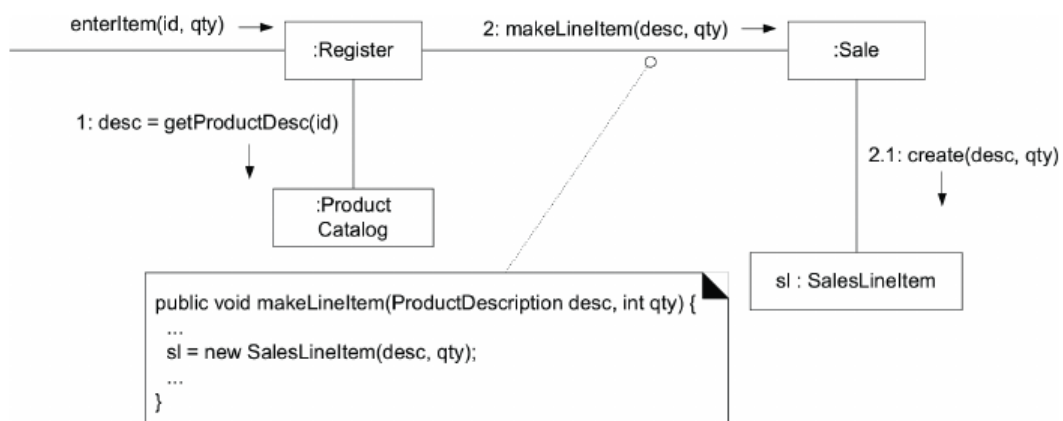
Questa è considerata una visibilità relativamente **permanente**, poichè il riferimento e il collegamento persistono per tutto il tempo in cui gli oggetti esistono.



9.2.2 Visibilità per parametro

Avviene quando l'oggetto B viene passato come parametro all'interno di un metodo dell'oggetto A.

Si tratta di una visibilità relativamente **temporanea**, in quanto è valida ed esiste esclusivamente per la durata dell'ambito del metodo in cui è stata passata



9.2.3 Visibilità locale

Si manifesta quando l'oggetto B è dichiarato come un oggetto locale all'interno di un metodo di A.

Anch'essa è una visibilità relativamente **temporanea** limitata all'ambito del metodo.

Può essere ottenuta in 2 modi:

- Creando una nuova istanza locale e assegnandola a una variabile
- Assegnando a una variabile locale l'oggetto restituito dalla chiamata a un altro metodo

9.2.4 Visibilità globale

Si ottiene quando l'oggetto B è dichiarato in modo da essere visibile globalmente ad A.

È una visibilità relativamente **permanente** e si realizza assegnando un'istanza a una variabile globale

9.3 Trasformazione della visibilità

La visibilità **non è un concetto statico**, ma le diverse forme di visibilità possono mutare ed essere convertite l'una nell'altra durante l'esecuzione nel codice.

9.3.1 Da visibilità locale a visibilità per parametro

Si verifica quando un oggetto istanziato o recuperato localmente all'interno di un metodo (visibilità locale) viene passato come argomento (parametro) nella chiamata a un altro metodo.

9.3.2 Da visibilità per parametro a visibilità per attributo

Da visibilità per parametro a visibilità per attributo: È una pratica comune nei metodi di inizializzazione, come i costruttori.

Un oggetto viene passato in ingresso a un metodo come parametro (visibilità temporanea) e, all'interno di quel metodo, viene assegnato a una variabile di istanza della classe, trasformandosi a tutti gli effetti in un attributo stabile nel tempo.

10. TDD - Test Driven Development

10.1 Definizione

È una pratica che prevede di scrivere il test unitario prima del codice di produzione.

Il programmatore scrive il test immaginando che **l'interfaccia esista già**, e solo successivamente scrive il codice per farlo passare.

10.2 Cosa è un test unitario?

Un test unitario è un test che si concentra sulla verifica di singole classi e metodi all'interno del codice.

10.2.1 Schema di test

1. **Preparazione:**
Si crea l'oggetto da verificare, chiamato anche fixture
2. **Esecuzione:**
Si eseguono specifiche azioni sulla fixture per attivare le operazioni che si desidera testare
3. **Verifica:**
Si controlla che i risultati ottenuti dal codice corrispondano ai risultati previsti
4. **Rilascio:**
Una fase opzionale in cui si ripuliscono o si rilasciano gli oggetti e le risorse che sono state utilizzate durante il test

10.2.2 Tipi di test

- Test di unità
- Test di integrazione
- Test di sistema
- Test di accettazione

10.3 Vantaggi

- **I test vengono effettivamente scritti:**
Obbligando a scrivere il test prima del codice, si ha la garanzia che i test non vengano tralasciati
- **Maggiore coerenza:**
La soddisfazione del programmatore nel far passare i test porta a una scrittura dei test più coerente e disciplinata
- **Chiarezza progettuale:**
Scrivere il test immaginando l'uso dell'oggetto aiuta a chiarire i dettagli dell'interfaccia e del comportamento atteso prima ancora di implementarli

- **Verifica continua:**

Si ottiene un sistema di verifica automatica, che è costantemente ripetibile e dimostrabile in qualsiasi momento

- **Fiducia nel cambiamento:**

La presenza di test automatici infonde fiducia nel programmatore quando è necessario modificare o fare refactoring del codice sapendo che eventuali errori verranno subito rilevati

10.4 Svantaggi

- Complessità di applicazione
- Curva di apprendimento ripida
- Maggiore tempo di sviluppo iniziale
- Aumento del codice da mantenere

10.5 Progettazione degli input

Per testare in modo efficiente, gli input vanno raggruppati in "partizioni di equivalenza" (insiemi di dati trattati allo stesso modo dal codice).

È fondamentale testare sia input validi che input deliberatamente sbagliati, prestando particolare attenzione ai valori ai limiti (o di frontiera), forzando errori o testando valori critici come puntatori nulli e zeri.

11. Refactoring

11.1 Definizione

È un **metodo strutturato** e disciplinato per riscrivere o ristrutturare del codice esistente senza modificarne il comportamento esterno.

Si tratta di un **processo iterativo** che procede applicando piccoli passi di trasformazione, costantemente supportati dalla ripetizione dei test dopo ciascuna modifica per assicurarsi di non aver "rotto" nulla.

11.2 Obiettivi e Vantaggi

L'utilità principale del refactoring è il miglioramento continuo del codice e la preparazione del sistema a futuri cambiamenti.

Tra le attività e gli scopi principali ci sono:

- Rimuovere il codice duplicato
- Migliorare la chiarezza e la leggibilità generale
- Abbreviare metodi troppo lunghi

11.3 Quando usarlo?

- Seguendo la "Regola del 3"
 1. **La prima volta** che scrivi una porzione di codice, limitati a scriverla per far funzionare la logica
 2. **La seconda volta** che hai bisogno di fare la stessa cosa, potresti semplicemente copiare e incollare il codice
 3. **La terza volta** che ti trovi a duplicare o implementare di nuovo quello stesso pezzo di codice, scatta la regola: devi fermarti e applicare il refactoring
- Si aggiunge una nuova funzionalità
- Si corregge un bug
- Durante la revisione del codice

11.4 Procedura

Il refactoring si esegue seguendo un ciclo preciso e ripetitivo:

1. **Assicurarsi che i test passino**, prima di toccare il codice, tutto deve funzionare
2. **Trovare un code smell**
3. **Determinare come migliorare** questo codice
4. **Effettuare le migliorie**

5. Eseguire nuovamente i test

6. Ripetere il ciclo

11.5 Cosa è un Code Smell?

Un Code Smell è un sintomo o un indicatore presente nel codice sorgente che suggerisce la necessità di applicare una tecnica di refactoring per migliorarne la progettazione.

Non si tratta di un bug o di un errore che impedisce l'esecuzione del programma, ma piuttosto di un difetto strutturale o di design che rende il software difficile da capire, da modificare e da riutilizzare nel tempo

11.6 Tabella Code Smell

Nome	Sintomi	Problema	Soluzione
Long Method	Un metodo contiene troppe righe di codice (spesso più di dieci).	I metodi lunghi sono molto difficili da capire, modificare, testare e riutilizzare.	Applicare <u>Extract Method</u> . Se ci sono troppe variabili locali, usare <u>Replace Temp with Query</u> o <u>Introduce Parameter Object</u> . Per i condizionali usare <u>Decompose Conditional</u> .
Duplicated Code	Lo stesso frammento di codice (o molto simile) appare in più punti del programma.	Se c'è un bug, va corretto in tutte le copie. Aumenta inutilmente la dimensione e la complessità del codice.	Applicare <u>Extract Method</u> per isolare il codice duplicato in un unico punto e sostituire i cloni con la chiamata a tale metodo.
Feature Envy	Un metodo accede continuamente ai dati di un'altra classe piuttosto che a quelli della propria.	Errore di progettazione: il metodo si trova nella classe sbagliata e viola il principio di incapsulamento.	Usare <u>Move Method</u> per spostare l'intero metodo, oppure <u>Extract Method</u> per isolare la parte "invidiosa" e spostarla nella classe che possiede i dati.
Large Class	Una classe contiene troppi campi, metodi o un numero eccessivo di linee di codice.	Violazione della coesione: la classe sta chiaramente gestendo più di una singola responsabilità.	Separare le responsabilità creando nuove classi mirate tramite <u>Extract Class</u> o, se ha senso gerarchicamente, <u>Extract Subclass</u> .
Switch Statement	Presenza di istruzioni <code>switch</code> o lunghe catene di <code>if-else</code> basate sul tipo di un oggetto.	La stessa struttura condizionale tende a ripetersi nel codice. Se si aggiunge un nuovo tipo, bisogna modificare tutti gli <code>switch</code> .	Sostituire la logica condizionale sfruttando il polimorfismo con <u>Replace Conditional with Polymorphism</u> .

Tabella 11.1

Nome	Sintomi	Problema	Soluzione
Data Class	Una classe contiene solo attributi e i rispettivi metodi getter/setter, senza logica.	È un contenitore passivo che spinge le altre classi a soffrire di <u>Feature Envy</u> , tipico di una programmazione procedurale e non a oggetti.	Cercare chi utilizza questi dati e migrare quelle funzionalità dentro la Data Class usando <u>Move Method</u> o <u>Extract Method</u> .
Long Parameter List	Un metodo richiede il passaggio di un numero eccessivo di parametri.	Firme dei metodi difficili da leggere e utilizzare, che tendono a cambiare spesso nel tempo.	Raggruppare i parametri correlati in un oggetto tramite <u>Introduce Parameter Object</u> , oppure passare l'intero oggetto con <u>Preserve Whole Object</u> .
Shotgun Surgery	Per completare una singola modifica funzionale, si devono fare tanti piccoli interventi sparsi in molte classi diverse.	Una singola responsabilità logica è stata frammentata e distribuita in modo errato su troppe classi.	Riunire i comportamenti correlati in un'unica classe usando <u>Move Method</u> e <u>Move Field</u> . Se alcune classi si svuotano, usare <u>Inline Class</u> .
Comment	Un blocco di codice o un metodo richiede un'enorme quantità di commenti per essere compreso.	Il codice non è auto-esplicativo e risulta scritto male. I commenti servono a compensare la scarsa chiarezza.	Estrarre le parti complesse con <u>Extract Variable</u> o <u>Extract Method</u> , assegnando nomi chiari (<u>Rename Method</u>) che rendano inutile il commento.